SG SisGera

DESIGN SYSTEM & GUIA DE ESTILO

Guia de Estilo — Frontend SisGera

Referência visual e de código para manter consistência entre telas: cores, tipografia, componentes, formulários e diretrizes de acessibilidade usadas no app.

Projeto: SisGera — Sistema de Gestão para Corpo de Bombeiros (TCC)
Repositório: sisgera-react (frontend) — fonte dos tokens: `src/styles/tokens.css`
Versão do documento: 1.0 — gerado a partir do código em produção

01 · FUNDAMENTOS

Cores

Todas as cores base ficam em `src/styles/tokens.css` como variáveis CSS. Nunca usar hex direto em componentes reutilizáveis — sempre `var(--nome-do-token)`. Isso é o que permite corrigir contraste ou tema em um único lugar e propagar para o app inteiro.

Marca e superfícies

Primária
`--cor-primaria · #D32F2F`

Primária (texto)
`--cor-primaria-texto · #c62828`

Fundo
`--cor-fundo · #f4f7fa`

Painel
`--cor-painel · #ffffff`

Texto principal
`--cor-texto-principal · #2d3748`

Texto secundário
`--cor-texto-secundario · #475569`

Borda
`--cor-borda · #e2e8f0`

Cores de status

Pendente
`--cor-pendente · #f59e0b`

Arquivado / Ativo
`--cor-arquivado · #10b981`

Validado
`--cor-validado · #3b82f6`

Rascunho
`--cor-rascunho · #9ca3af`

Urgente (fundo)
`--cor-urgente-bg · #fee2e2`

Urgente (texto)
`--cor-urgente-text · #991b1b`

> **Regra de contraste:** todo texto precisa de no mínimo **4.5:1** de contraste contra o fundo real (WCAG AA). Cuidado com dois erros já cometidos neste projeto: (1) aplicar `opacity` num texto que já tinha cor correta — a opacidade mistura com o fundo e derruba o contraste calculado; (2) usar a cor da marca como *texto* sobre um fundo colorido claro (ex: badge, link ativo) — nesse caso use `--cor-primaria-texto`, não `--cor-primaria`.

Modo escuro

O dark mode redefine as mesmas variáveis dentro de `body.dark-mode` — os componentes não sabem que o tema mudou, só o valor da variável muda.

```
body.dark-mode {
  --cor-fundo: #0f172a;
  --cor-painel: #1e293b;
  --cor-texto-principal: #f8fafc;
  --cor-texto-secundario: #cbd5e1;
  --cor-borda: #334155;
}
```

01 · FUNDAMENTOS

Tipografia, espaçamento e elevação

Família e escala tipográfica

Fonte do sistema operacional (sem webfont carregada) — carrega instantâneo, sem FOUT:

`-apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif`

USO	TAMANHO / PESO	EXEMPLO
Título de página (`.main-header h2`)	28px / 700	**Controle de Ocorrências**
Título de card/modal (`h3`)	18px / 600	Dados do Produto
Cabeçalho de seção de form (`.section-header`)	15px / 700	Sinais Vitais
Corpo / label	13–14px / 500–600	Nome Completo *
Texto secundário / meta	12–13px / 500, cor secundária	Preencha os dados abaixo...
Cabeçalho de tabela (`th`)	12px / 600, uppercase	STATUS
Badge / pill	11–12px / 600	Validado

Raio de borda

6px
`botões outline, dropdown`

8px
`inputs, botões, chips`

12px
`painéis, cards, modais`

9999px
`badges, pills, avatar`

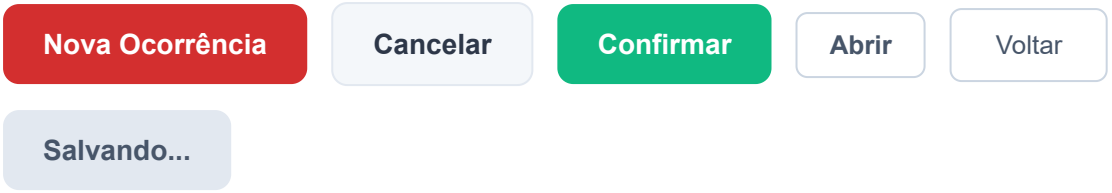
Elevação (sombra)

TOKEN / USO	VALOR
`--sombra-padrao` — painéis	`0 4px 6px -1px rgba(0,0,0,.1), 0 2px 4px -2px rgba(0,0,0,.1)`
KPI card	`0 4px 6px -1px rgba(0,0,0,.05), 0 2px 4px -1px rgba(0,0,0,.03)` (mais sutil)
Modal	`0 20px 60px rgba(0,0,0,.25)`
Dropdown	`0 4px 6px -1px rgba(0,0,0,.1), 0 2px 4px -1px rgba(0,0,0,.06)`

02 · COMPONENTES

Botões

Definidos em `src/styles/base.css` (`.btn-primary`) e `src/styles/components.css` (demais variantes). Use sempre a classe pronta — não recriar estilo de botão inline.



CLASSE	QUANDO USAR
`.btn-primary`	Ação principal da tela (1 por contexto) — vermelho da marca
`.btn-secondary`	Ação neutra ao lado da primária (ex: "Cancelar")
`.btn-success`	Confirmação positiva explícita (raro — preferir `.btn-primary`)
`.btn-outline` / `.btn-outline-secondary`	Ações secundárias em listas/tabelas, links estilizados como botão
`:disabled`	Estado de carregamento/bloqueado — nunca remover o botão, desabilitar

02 · COMPONENTES

Badges e indicadores de status

Existem duas famílias com o mesmo propósito visual — usar a certa conforme o contexto para não divergir cores entre telas parecidas.

`.status-badge` — Produtos, Patrimônio, Doações (components.css)

Pendente Urgente Arquivado Validado Rascunho Inativo Em Manutenção

`.status-pill` — Ocorrências (Ocorrencias.css)

Pend. Validação Rascunho Em Andamento Arquivado Cancelado Pend. Arquivamento

> Se um novo módulo precisar de status, prefira reaproveitar `.status-badge` (é o padrão mais usado) em vez de inventar uma terceira variante de badge.

02 · COMPONENTES

Cards

KPI Card (`.kpi-card`) — usado em Dashboard e listagens

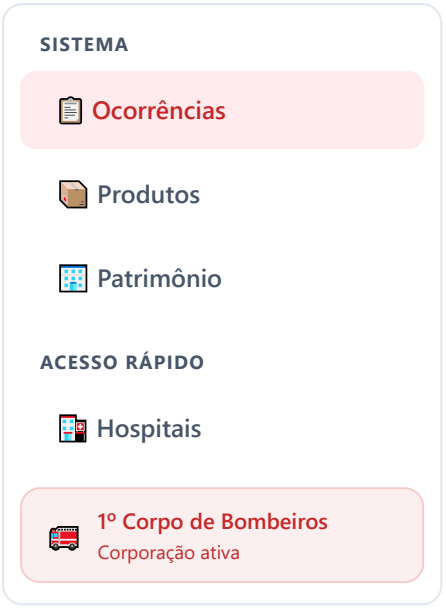


Painel (`.panel`)

Container base para tabelas e seções — fundo `--cor-painel`, radius 12px, `--sombra-padrao`. É o "cartão" que envolve praticamente todo conteúdo de listagem no app.

Sidebar (`components/Sidebar`)

Único componente, importado uma vez em `App.js` — nunca duplicar sidebar em uma página específica.



02 · COMPONENTES

Formulários

Padrão base em `.form-group` (components.css). **Todo `<label>` deve ter `htmlFor` apontando pro `id` do campo** — isso já foi motivo de correção de acessibilidade neste projeto (labels visuais sem associação real ao input).

Nome Completo *

Ex: Maria Silva

Tipo de Ocorrência

Simplificado

Data de Nascimento *

15/03/1999

CPF *

123.456

CPF inválido — informe os 11 dígitos.

Chips e toggle-cards

Via pública

✓ Rodovia

```
<div className="form-group">
  <label htmlFor="data_ocorrencia">Data da Ocorrência *</label>
  <input type="date" id="data_ocorrencia" value={...} onChange={...} />
</div>
```

Checklist de acessibilidade ao criar um formulário novo

- ✓ Todo `<label>` tem `htmlFor` = `id` do campo (campos repetidos em lista: gerar id único com o `uid` do item)
- ✓ Botões só-ícone têm `aria-label` (fechar modal, remover item, buscar)
- ✓ `<select>` sem label visível tem `aria-label`
- ✓ Headings seguem ordem sequencial (h1→h2→h3...) mesmo quando conteúdo é condicional/por passo
- ✓ Contraste de texto ≥ 4.5:1 — não usar `opacity` pra "suavizar" texto, usar uma cor com contraste já calculado

02 · COMPONENTES

Tabelas, modais e stepper

Tabela (`.data-table`)

LOCAL	TIPO	STATUS
Almenara / Centro	Resgate	Rascunho
BR-116, Km 42	Simplificado	Arquivado

Modal (`.modal-container`)



Stepper (formulário de 4 passos — Nova Ocorrência)



> Cada passo (`.form-step`) só existe no DOM quando ativo (`display:none` nos demais). Atenção: isso significa que os headings dos outros passos **não contam** para a ordem de heading da página enquanto escondidos — todo passo precisa começar num heading que não pule nível a partir do título do modal.

03 · MANUTENÇÃO

Convenções para alterações futuras

Onde mexer em cada coisa

ARQUIVO	CONTÉM
`src/styles/tokens.css`	Única fonte de cores/variáveis — `:root` (claro) e `body.dark-mode` (escuro)
`src/styles/base.css`	Reset, layout principal, `.btn-primary`, utilitários (`.text-center` etc.)
`src/styles/components.css`	Componentes reutilizáveis entre páginas: botões, badges, tabelas, modais, formulário genérico
`src/styles/dark-mode.css`	Overrides pontuais de dark mode que não dá pra resolver só com variável
`src/pages/<Modulo>/<Modulo>.css`	Estilo exclusivo daquela página — não duplicar o que já existe em components.css
`src/components/<Nome>/`	Componente React reutilizável entre páginas (ex: Sidebar, StatCard)

Antes de adicionar uma cor nova

- ✓ Já existe um token parecido em `tokens.css`? Reaproveitar em vez de criar outro hex
- ✓ Calcular contraste contra o fundo real onde o texto vai aparecer (não contra branco por padrão — cards coloridos, badges e sidebar têm fundos diferentes)
- ✓ Se for variante "texto sobre fundo tingido" (ex: badge, link ativo), considerar um token `-texto` dedicado em vez de reusar a cor de marca/ícone direto (ver `--cor-primaria-texto`)
- ✓ Definir também o valor equivalente em `body.dark-mode`

Antes de abrir um PR de UI

- ✓ `npm run build` local sem erros
- ✓ Rodar Lighthouse (aba Accessibility) na página alterada, em todos os passos/estados relevantes (não só o estado inicial)
- ✓ Conferir modo claro e escuro
- ✓ Se mexeu em `tokens.css` ou `components.css`: o efeito é global — checar outras páginas que usam a mesma classe/variável

> Este documento é gerado a partir do código real (`src/styles/*.css`) — se um valor aqui divergir do código, o código é a fonte da verdade. Atualize este arquivo (`docs/style-guide.html`) e re-exporte o PDF sempre que os tokens principais mudarem.